**Task 1: Watch the Terraform-03 video.**
**Task 2: Execute the script shown in the video.**

# 1. Objective

To understand and implement different methods of passing input variables to Terraform configurations using an existing Terraform project. The lab demonstrates how variable values can be supplied through default values, command-line arguments, environment variables, and variable files.

# 2. Steps Used

## Step 1: Review the Existing Terraform Configuration

Used the existing project directory:

```
C:\repos\Terraform-01
```

The project contained two resources:

- `local_file.asif_pet` — creates/manages the local file `asif-runtime.txt`.
- `random_pet.brownie` — generates a random pet name using a configurable prefix.

The `local_file.asif_pet` resource had `lifecycle.prevent_destroy = true`, which protects it from being destroyed by Terraform.

## Step 2: Verify the Initial Configuration

Executed:

```
terraform plan
```

Terraform initially detected a content mismatch between the configured value and the existing file. Because `prevent_destroy` was enabled, Terraform blocked the replacement.

Updated the `content` default in `variables.tf` to match the existing file content:

```
variable "content" {
  default = "Original content for Asif"
}
```

After correcting the configuration, `terraform plan` reported no changes.

## Step 3: Method 1 — Default Variable Values

Configured the `prefix` variable in `variables.tf` with the default value `default_method`.

```
variable "prefix" {
  default = "default_method"
}
```

Executed:

```
terraform plan
terraform apply
```

Confirmed the changes by approving the apply operation.

**Result:** Terraform replaced the `random_pet.brownie` resource using the configured default prefix.

Generated ID: `default_method.rightly.amusing.monarch`

## Step 4: Method 2 — Command-Line Variable

Executed:

```
terraform plan -var="prefix=interactive"
terraform apply -var="prefix=interactive"
```

Reviewed the plan and approved the apply operation.

**Result:** Terraform used the value supplied through `-var`, overriding the default value.

Generated ID: `interactive.formally.grown.wolf`

*Note: This method uses a command-line variable override, not interactive variable prompting.*

## Step 5: Method 3 — Environment Variable

Set the environment variable in PowerShell:

```
$env:TF_VAR_prefix = "environment"
```

Executed:

```
terraform plan
terraform apply
```

Approved the apply operation.

**Result:** Terraform read the variable from the `TF_VAR_prefix` environment variable.

Generated ID: `environment.luckily.concrete.gull`

## Step 6: Method 4 — Explicit `.tfvars` File

Created `variables.tfvars` with the following content:

```
prefix = "tfvars"
```

Executed:

```
terraform plan -var-file="variables.tfvars"
terraform apply -var-file="variables.tfvars"
```

Approved the apply operation.

**Result:** Terraform used the value supplied by the explicitly selected variable file.

Generated ID: `tfvars.neatly.clever.wren`

## Step 7: Method 5 — Automatic Variable File Loading

Created a file named `terraform.tfvars` containing:

```
prefix = "auto_tfvars"
```

Executed:

```
terraform plan
terraform apply
```

Approved the apply operation.

**Result:** Terraform automatically loaded `terraform.tfvars` without requiring the `-var-file` option.

Generated ID: `auto_tfvars.jointly.smart.slug`

## Step 8: Cleanup and Final Verification

Removed the temporary environment variable:

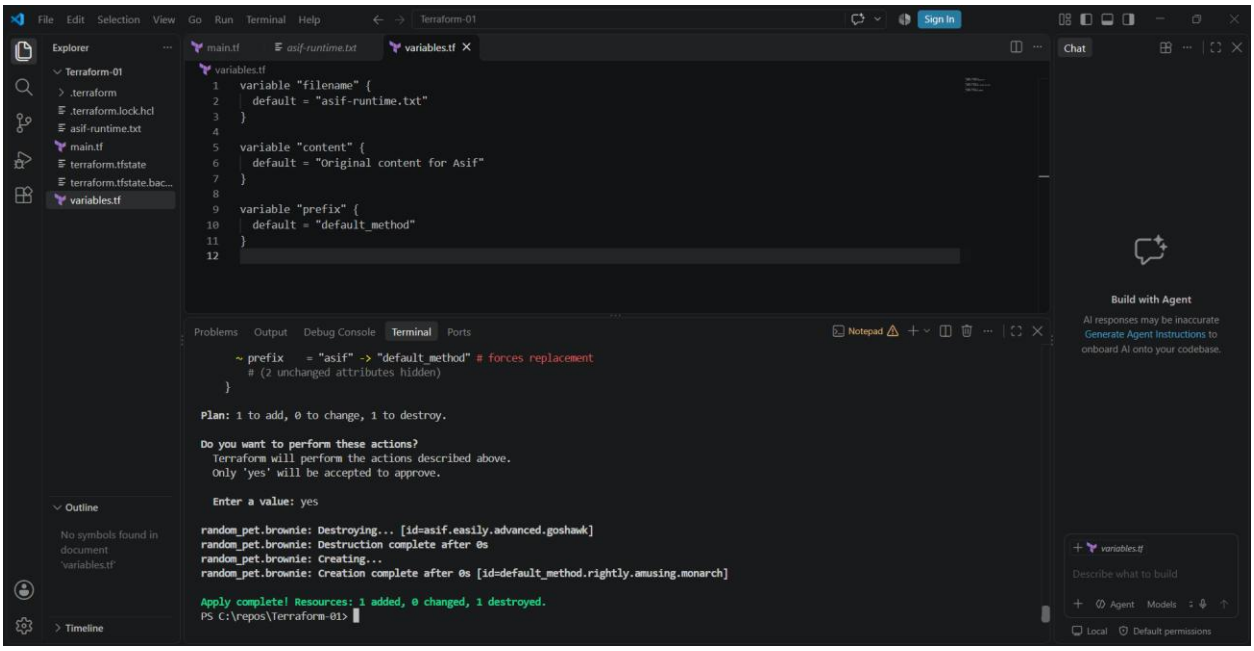
```
Remove-Item Env:TF_VAR_prefix -ErrorAction SilentlyContinue
```

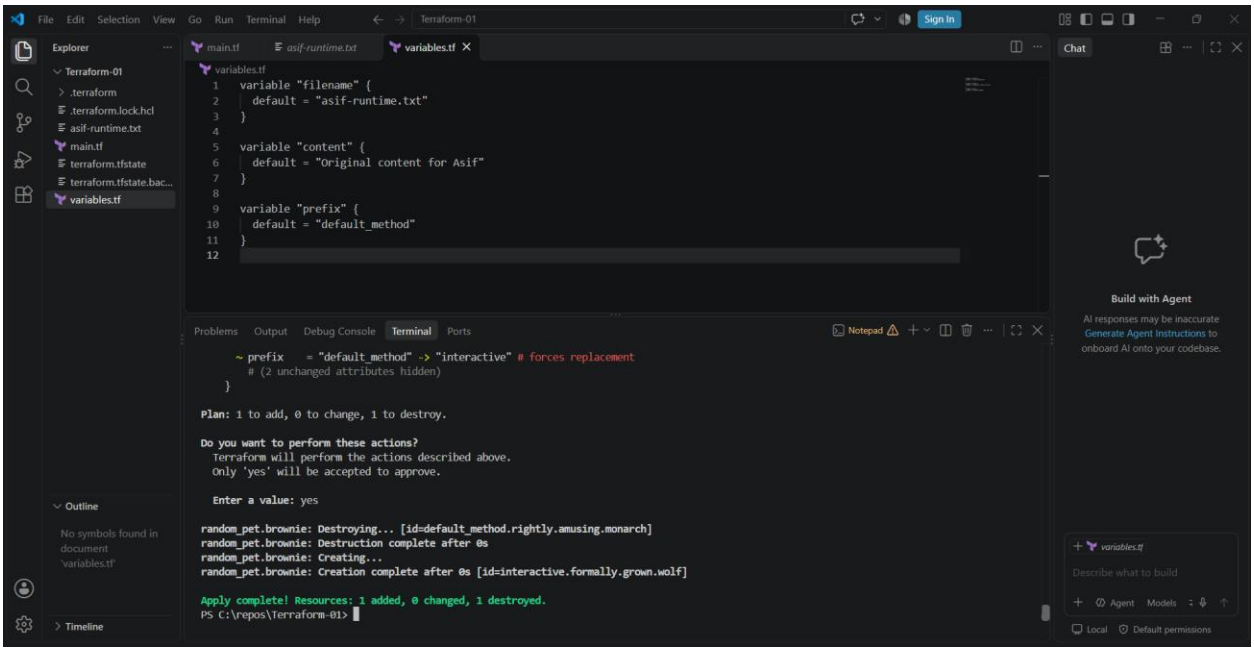
Executed the final verification:

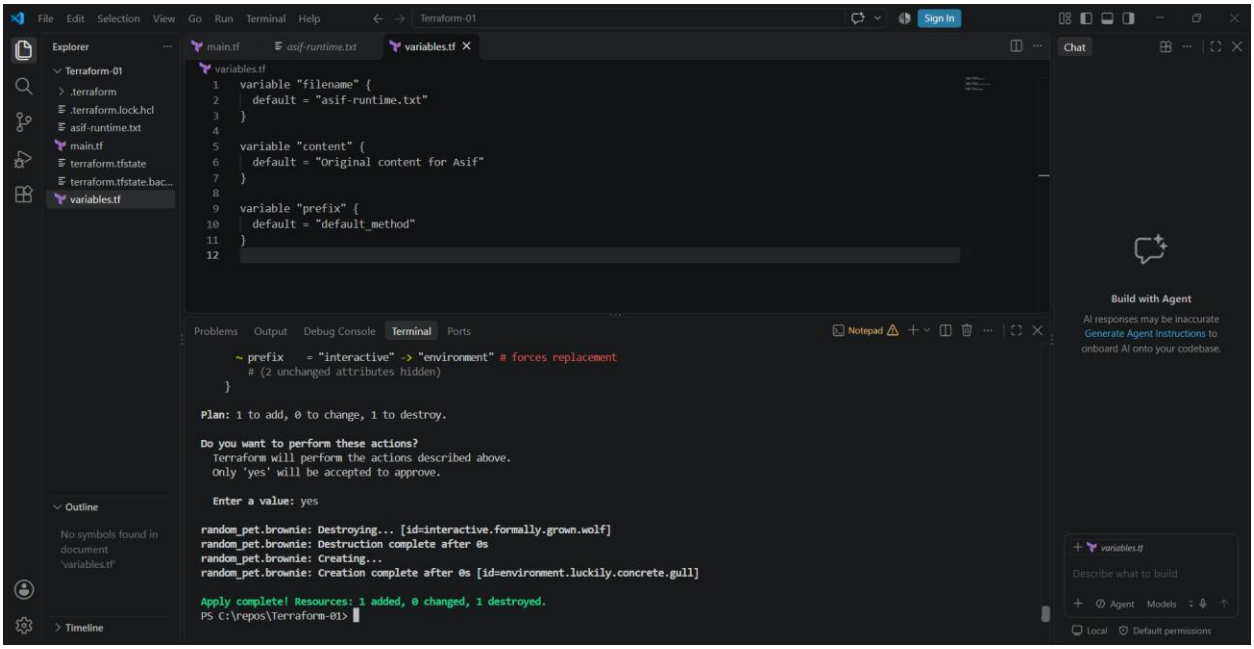
```
terraform plan
```

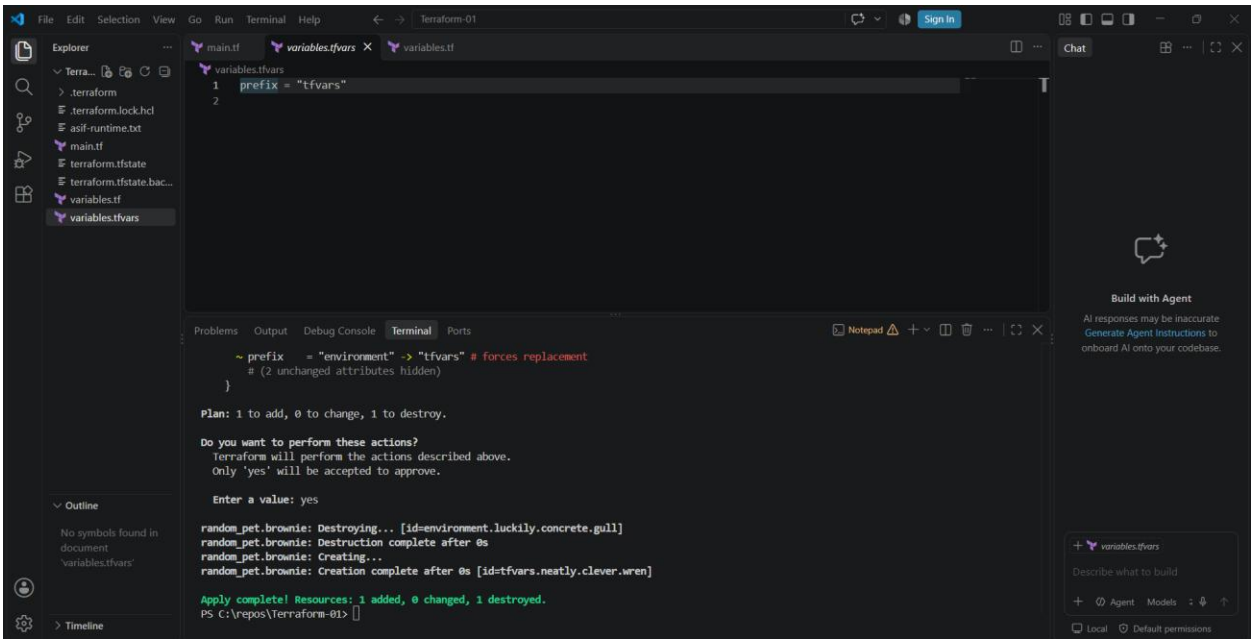
**Result:** Terraform reported:

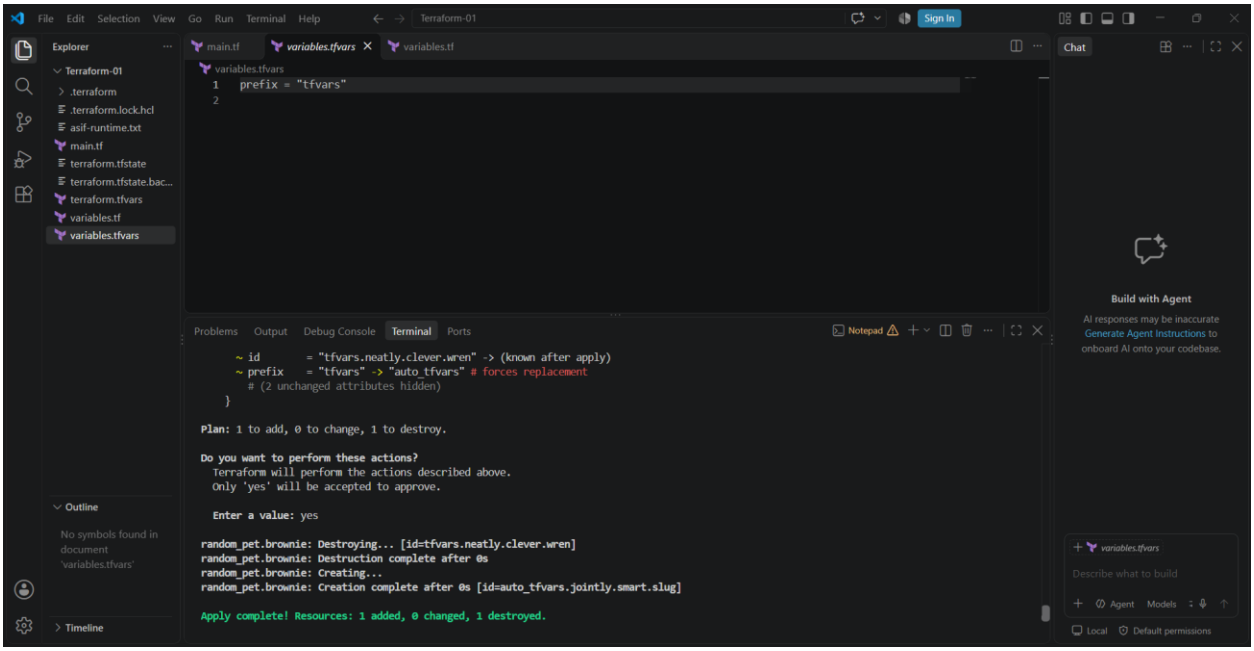
```
No changes. Your infrastructure matches the configuration.
```




File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif-runtime.txt
main.tf
terraform.tfstate
terraform.tfstate.bac...
variables.tf
main.tf
asif-runtime.txt
variables.tf
variables.tf
1 variable "filename" {
2 default = "asif-runtime.txt"
3 }
4
5 variable "content" {
6 default = "Original content for Asif"
7 }
8
9 variable "prefix" {
10 default = "default_method"
11 }
12
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
Notepad
~ prefix = "default_method" -> "interactive" # forces replacement
# (2 unchanged attributes hidden)
}
Plan: 1 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.brownie: Destroying... [id=default_method.rightly.amusing.monarch]
random_pet.brownie: Destruction complete after 0s
random_pet.brownie: Creating...
random_pet.brownie: Creation complete after 0s [id=interactive.formally.grown.wolf]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\repos\Terraform-01>
Outline
No symbols found in document 'variables.tf'
Timeline
variables.tf
Describe what to build
Agent Models
Local Default permissions


File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif-runtime.txt
main.tf
terraform.tfstate
terraform.tfstate.bac...
variables.tf
main.tf
asif-runtime.txt
variables.tf
variables.tf
1 variable "filename" {
2 default = "asif-runtime.txt"
3 }
4
5 variable "content" {
6 default = "Original content for Asif"
7 }
8
9 variable "prefix" {
10 default = "default_method"
11 }
12
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
Notepad
~ prefix = "interactive" -> "environment" # forces replacement
# (2 unchanged attributes hidden)
}
Plan: 1 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.brownie: Destroying... [id=interactive.formally.grown.wolf]
random_pet.brownie: Destruction complete after 0s
random_pet.brownie: Creating...
random_pet.brownie: Creation complete after 0s [id=environment.luckily.concrete.gull]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\repos\Terraform-01>
Outline
No symbols found in document 'variables.tf'
Timeline
variables.tf
Describe what to build
Agent Models
Local Default permissions




# 3. Observation

1. Terraform supports multiple methods for supplying input variable values.
2. Default values are used when no higher-precedence value is supplied.
3. The `-var` option overrides the default variable value.
4. Environment variables prefixed with `TF_VAR_` can supply Terraform input values.
5. The `-var-file` option allows a specific variable file to be selected.

6. Terraform automatically loads `terraform.tfvars` and supported `*.auto.tfvars` files.
7. The `random_pet.brownie` resource was replaced each time the prefix changed because the prefix is part of the resource configuration.
8. The existing local file was preserved because its configured content was aligned with the existing content and `prevent_destroy = true` remained enabled.
9. The final `terraform plan` reported no changes, confirming that the configuration and Terraform state were synchronized.

## 4. Conclusion

Successfully demonstrated five ways of providing Terraform input variable values: default values, command-line overrides, environment variables, explicitly selected variable files, and automatically loaded variable files.

Observed how changing a variable can trigger a resource replacement and how Terraform's lifecycle protection prevents the destruction of a protected resource. Final verification confirmed that the configuration matched the current Terraform state.

# Task 3: Integrate Terraform in Jenkins using the Terraform plugin.

## 1. Objective

To integrate Terraform with Jenkins using a Declarative Pipeline and automate Terraform initialization, validation, planning, application, and output verification.

The pipeline uses an isolated working directory to prevent interference with Terraform state from other projects.

## 2. Steps Used

1. Opened the Jenkins dashboard and selected the `terraform-pipeline-lab` job.
2. Configured the job using **Pipeline script**.
3. Checked the installed Terraform version using `terraform -version`.
4. Created the Terraform configuration file `main.tf` inside the `terraform-lab` directory.
5. Initialized Terraform using `terraform init -input=false`.
6. Validated the configuration using `terraform validate`.

7. Generated and saved the execution plan using `terraform plan -input=false -out=tfplan`.
8. Applied the saved plan using `terraform apply -input=false -auto-approve tfplan`.
9. Verified the generated output file and its contents.
10. Confirmed that the Jenkins build finished with `SUCCESS`.

## 3. Pipeline Code

```
pipeline {
    agent any

    stages {
        stage('Terraform Version') {
            steps {
                sh 'terraform -version'
            }
        }

        stage('Create Terraform Configuration') {
            steps {
                dir('terraform-lab') {
                    writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "jenkins_output" {
  filename = "${path.module}/jenkins-terraform-output.txt"
  content  = "Terraform executed successfully from Jenkins pipeline."
}
'''
                    sh 'cat main.tf'
                }
```

```
            }
        }

        stage('Terraform Init') {
            steps {
                dir('terraform-lab') {
                    sh 'terraform init -input=false'
                }
            }
        }

        stage('Terraform Validate') {
            steps {
                dir('terraform-lab') {
                    sh 'terraform validate'
                }
            }
        }

        stage('Terraform Plan') {
            steps {
                dir('terraform-lab') {
                    sh 'terraform plan -input=false -out=tfplan'
                }
            }
        }

        stage('Terraform Apply') {
            steps {
                dir('terraform-lab') {
                    sh 'terraform apply -input=false -auto-approve
tfplan'
                }
            }
        }

        stage('Verify Output') {
            steps {
                dir('terraform-lab') {
```

```
                    sh '''
                        cat jenkins-terraform-output.txt
                        test -f jenkins-terraform-output.txt
                    '''
                }
            }
        }
    }

    post {
        success {
            echo 'Isolated Terraform pipeline completed successfully!'
        }
        failure {
            echo 'Pipeline failed. Check Console Output.'
        }
    }
}
```

# 4. Explanation of the Pipeline

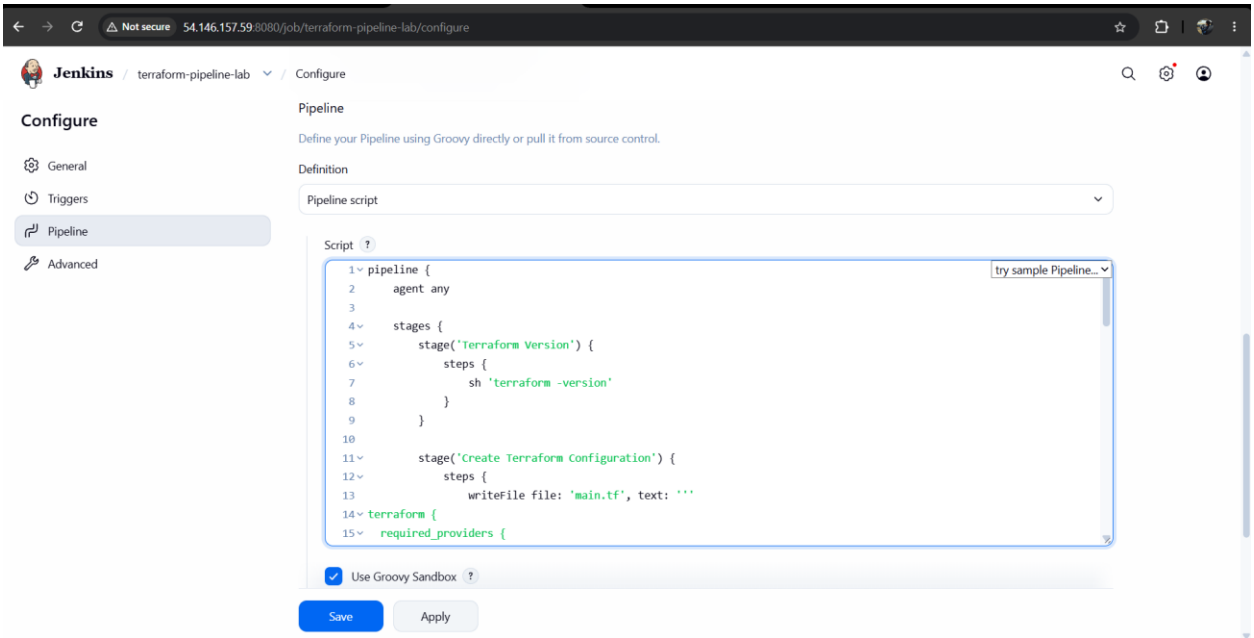
| Stage | Purpose |
|---|---|
| Terraform Version | Checks the installed Terraform version. |
| Create Terraform Configuration | Creates `main.tf` automatically from Jenkins. |
| Terraform Init | Downloads the required provider and initializes the working directory. |
| Terraform Validate | Checks whether the Terraform configuration is valid. |
| Terraform Plan | Previews the proposed changes and saves the plan to `tfplan`. |
| Terraform Apply | Executes the saved plan and creates the resource. |
| Verify Output | Checks that the output file exists and displays its contents. |

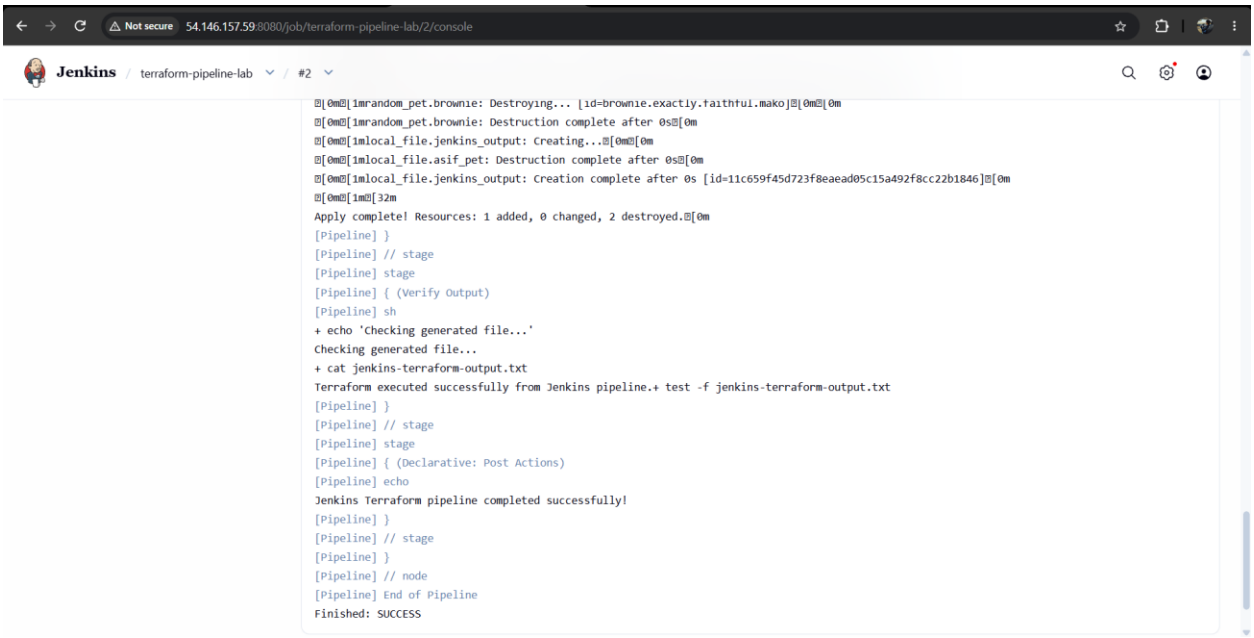
## Why use `dir('terraform-lab')`?

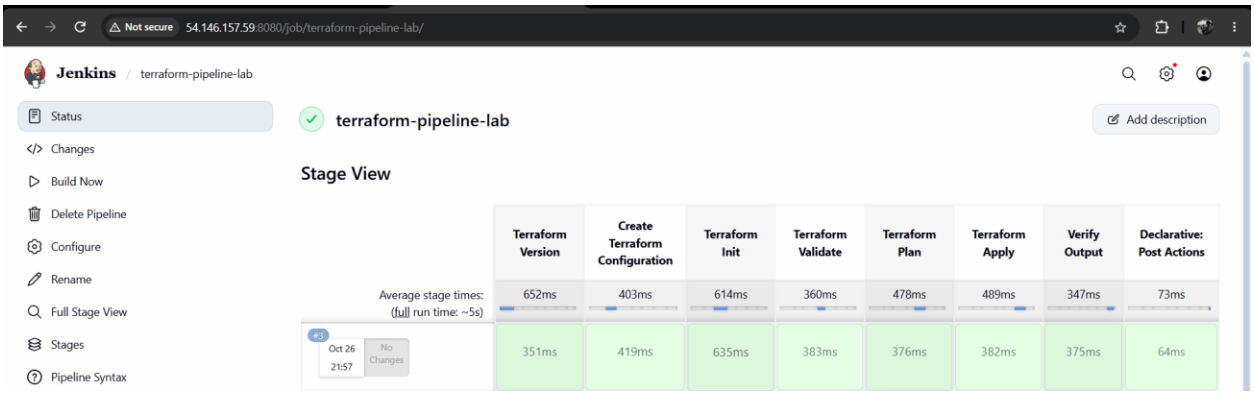
The `dir()` step makes the enclosed commands execute inside the specified Jenkins workspace subdirectory. Terraform therefore uses a separate state file in `terraform-lab`, isolating this lab from the root workspace's Terraform state.

## Why use `-auto-approve`?

It allows Terraform to apply the plan without waiting for interactive approval. In production, approval controls may be required before applying infrastructure changes.






# 5. Observation

- Terraform version: **v1.17.0**
- Local provider: **v2.9.1**
- Terraform initialization completed successfully.
- Configuration validation returned `Success!`.
- The execution plan showed **1 resource to add, 0 to change, and 0 to destroy**.
- Terraform created `local_file.jenkins_output`.
- The generated file contained:

```
Terraform executed successfully from Jenkins pipeline.
```

- The file verification passed.
- Final Jenkins build status: **SUCCESS**.

# 6. Conclusion

Successfully integrated Terraform with Jenkins through a Declarative Pipeline. The pipeline automated the complete Terraform workflow, from configuration creation to resource verification. Using a dedicated working directory isolated the lab's Terraform state and prevented the pipeline from modifying resources tracked by the previous root workspace state.

**Result:** The Jenkins–Terraform pipeline executed successfully, and the generated output file was verified.

**Task 4: Create one Jenkins job using Maven Project for the code below with two stages:**

- **Stage 1:** Git clone
- **Stage 2:** Maven Compilation **Code:** https://github.com/betawins/java-Working-app.git

# 1. Objective

To create a Jenkins Maven Project job that automatically clones Java application source code from GitHub and compiles it using Maven.

# 2. Steps I Used

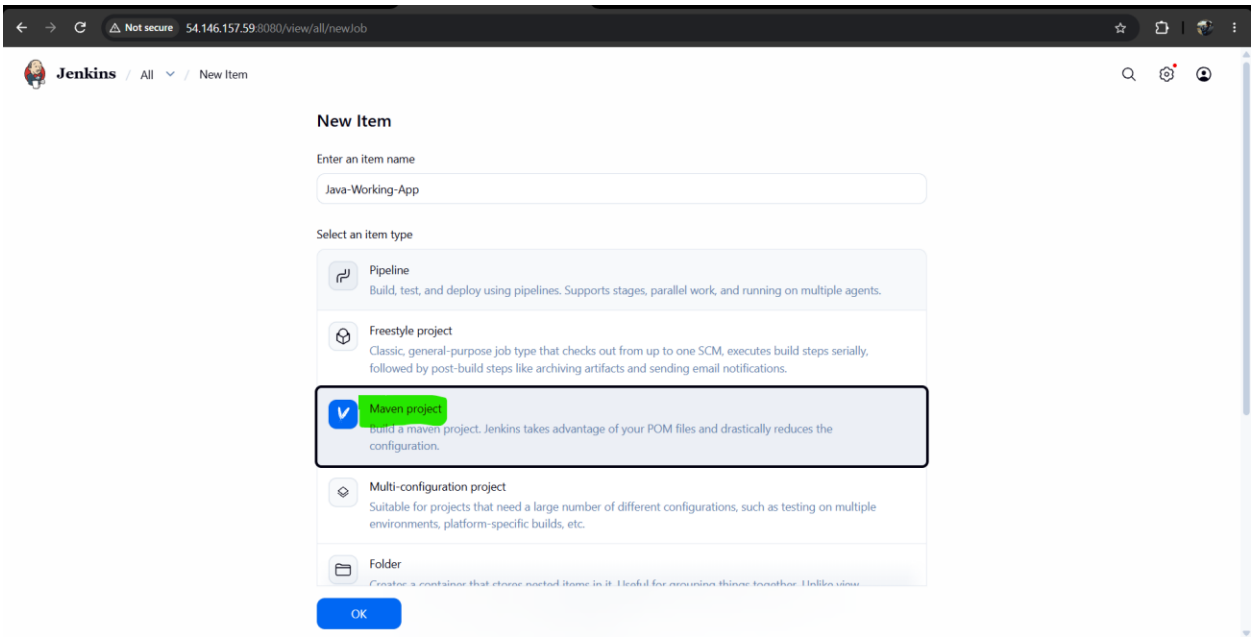
1. Opened the Jenkins dashboard.
2. Installed the Maven Integration plugin (version 3.28) because the Maven Project option was not available initially.
3. Created a new Maven Project job named `Java-Working-App`.
4. Configured Source Code Management as Git.
5. Added the repository URL:

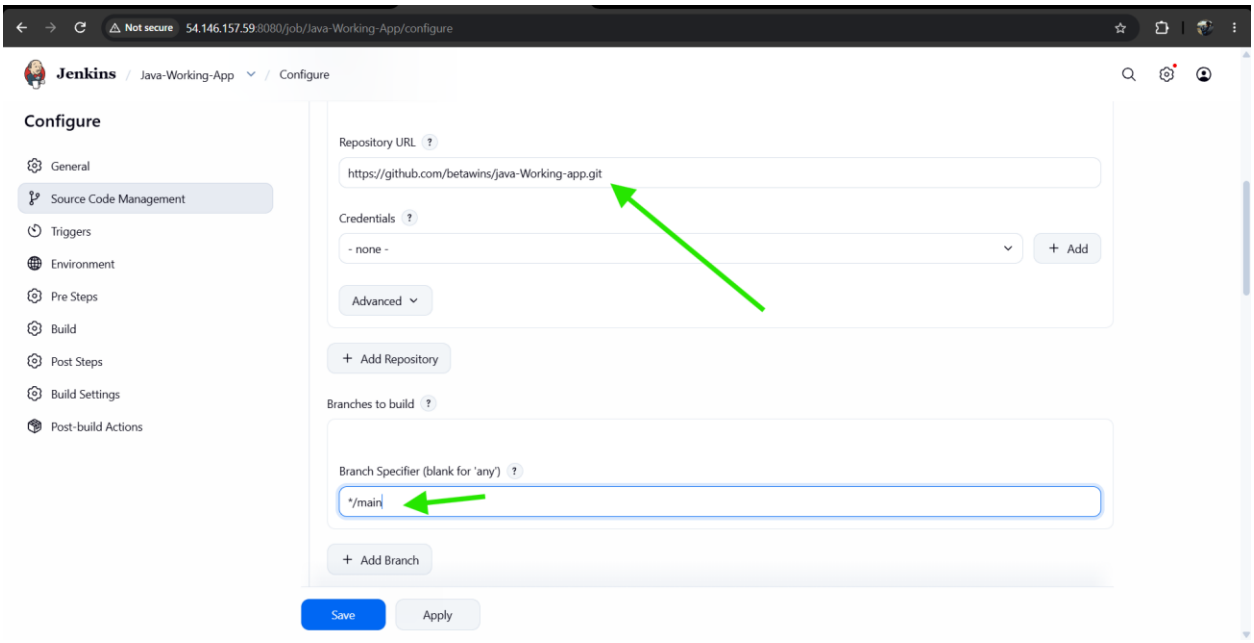
https://github.com/betawins/java-Working-app.git

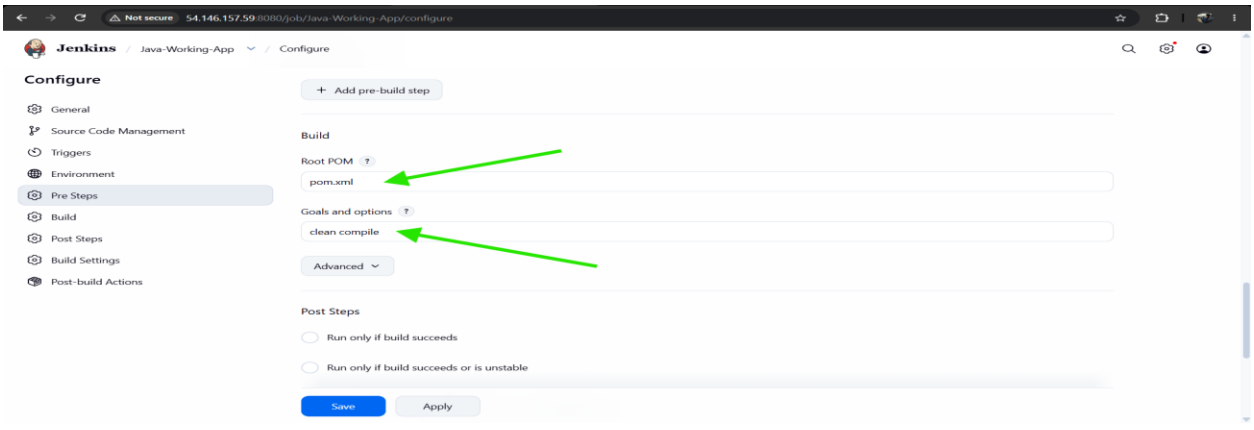
6. Configured the branch as `*/main`.
7. Set the Root POM to `pom.xml`.
8. Configured Maven goals as:

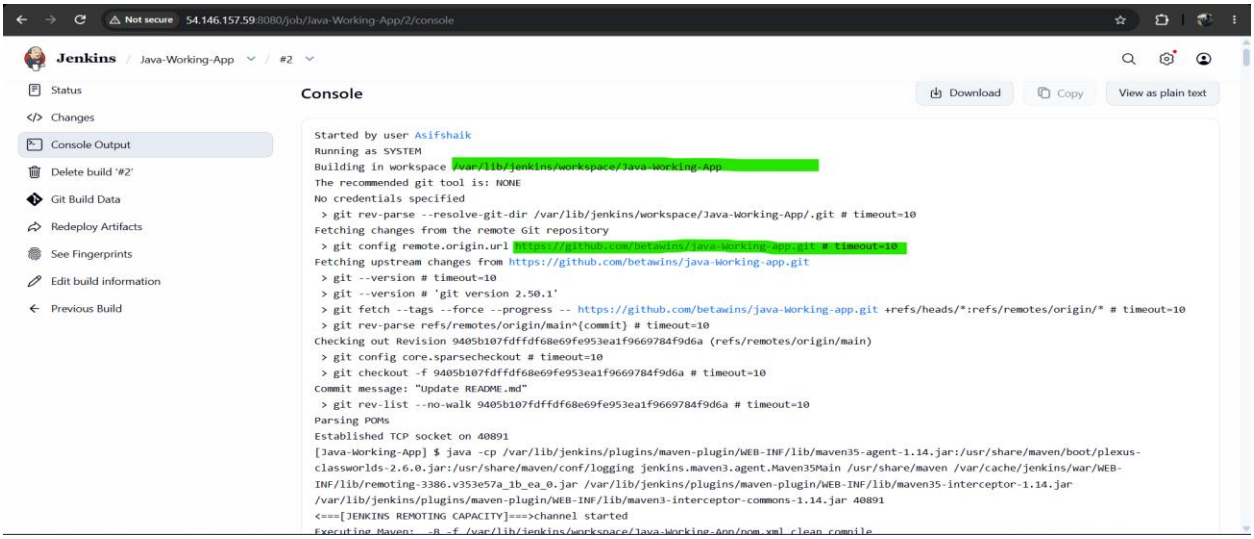
```
clean compile
```

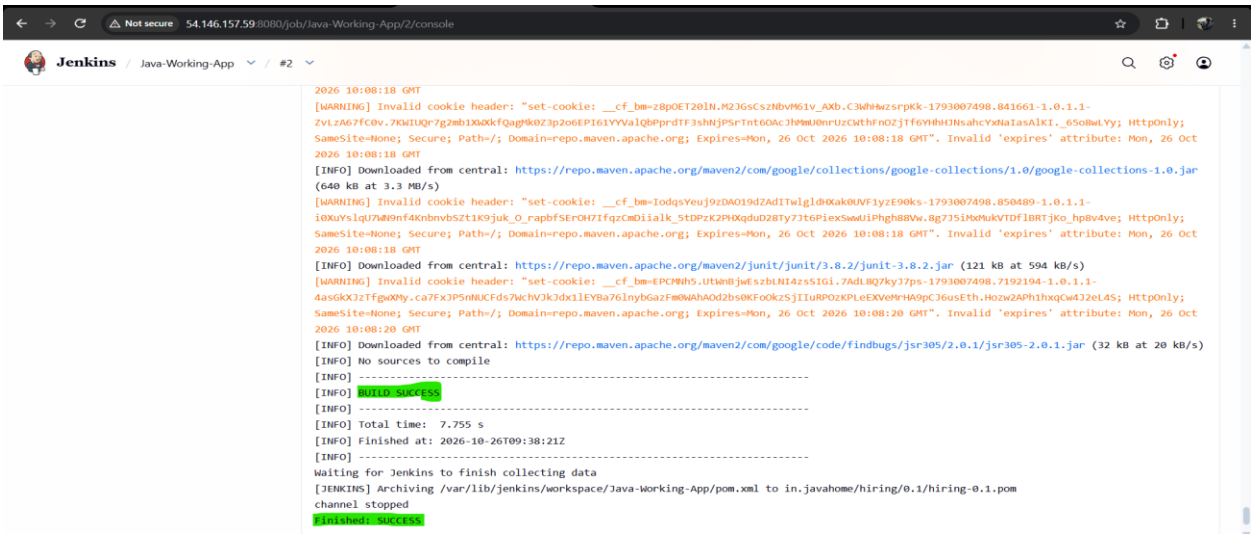
9. Saved the job and clicked Build Now.
10. Opened Console Output to check the build logs and verify the result.


Not secure 54.146.157.59:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Java-Working-App
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
OK


Not secure 54.146.157.59:8080/job/Java-Working-App/configure
Jenkins / Java-Working-App / Configure
Configure
General
Source Code Management
Triggers
Environment
Pre Steps
Build
Post Steps
Build Settings
Post-build Actions
Repository URL
https://github.com/betawins/java-Working-app.git
Credentials
- none -
+ Add
Advanced
+ Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
+ Add Branch
Save
Apply




# 3. Observation

- Jenkins successfully cloned the GitHub repository.

- The project POM was detected, and Maven downloaded the required dependencies.
- Maven completed the compilation process successfully.
- The `pom.xml` was archived by Jenkins.
- The final build status was SUCCESS.

## 4. Troubleshooting I Did

| Issue | Troubleshooting performed |
|---|---|
| Maven Project option was missing | Installed the Maven Integration plugin. |
| Initial build failed with `java.lang.UnsupportedOperationException` in the Maven launcher | Checked the Java and Maven versions on the Jenkins server and adjusted the Jenkins Maven installation configuration to use `/usr/share/maven`. |
| Maven displayed `Invalid cookie header` warnings | Checked the final build result. The warnings did not stop the build. |

Note: The initial Maven launcher error was resolved by the subsequent configuration/testing process; the successful build log confirms the final result.

## 5. Conclusion

I successfully configured and executed a Jenkins Maven Project for a Java application. Jenkins retrieved the source code from GitHub and executed the Maven `clean compile` goals. I verified the result through Console Output, which displayed `BUILD SUCCESS` and `Finished: SUCCESS`.

**Task 5: Use the same code and create a parameterized job in Jenkins with:**

- **Stage 1:** Git clone
- **Stage 2:** Maven Compilation **Code:** https://github.com/betawins/java-Working-app.git

# 1. Objective

To create a parameterized Jenkins Maven job that retrieves a Java application from GitHub, accepts a user-defined application name through a build parameter, and compiles the application using Maven.

# 2. Steps Used

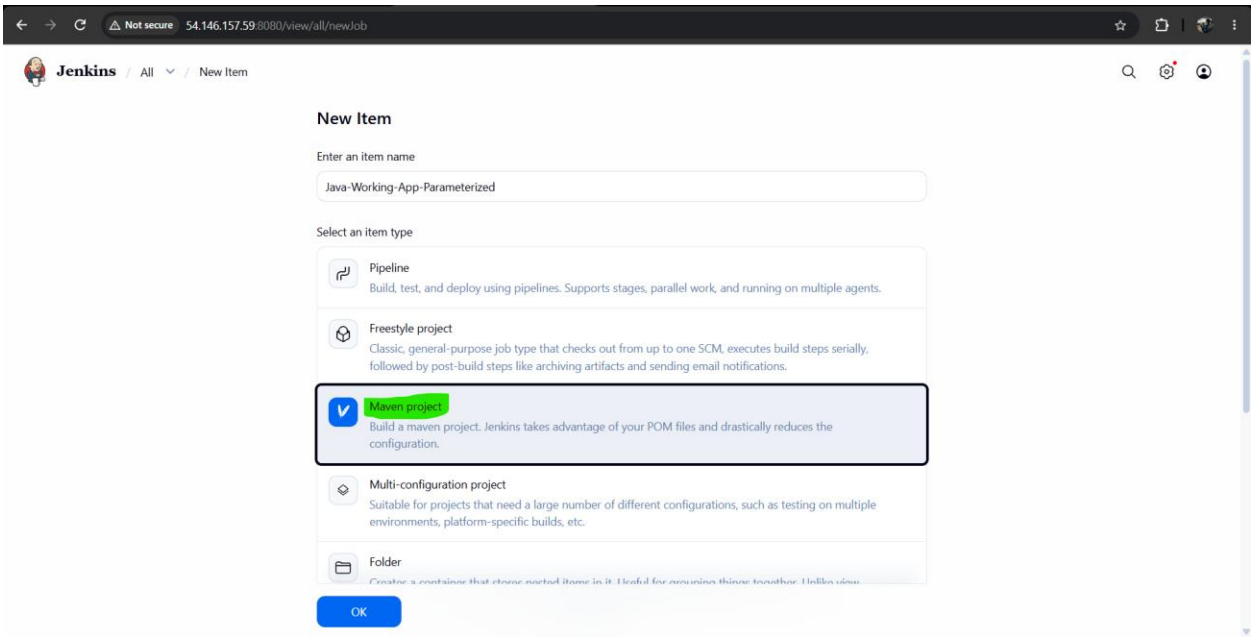
1. Opened the Jenkins dashboard and clicked New Item.
2. Created a new job named `Java-Working-App-Parameterized` and selected Maven Project.
3. Configured Source Code Management by selecting Git and providing the repository URL:

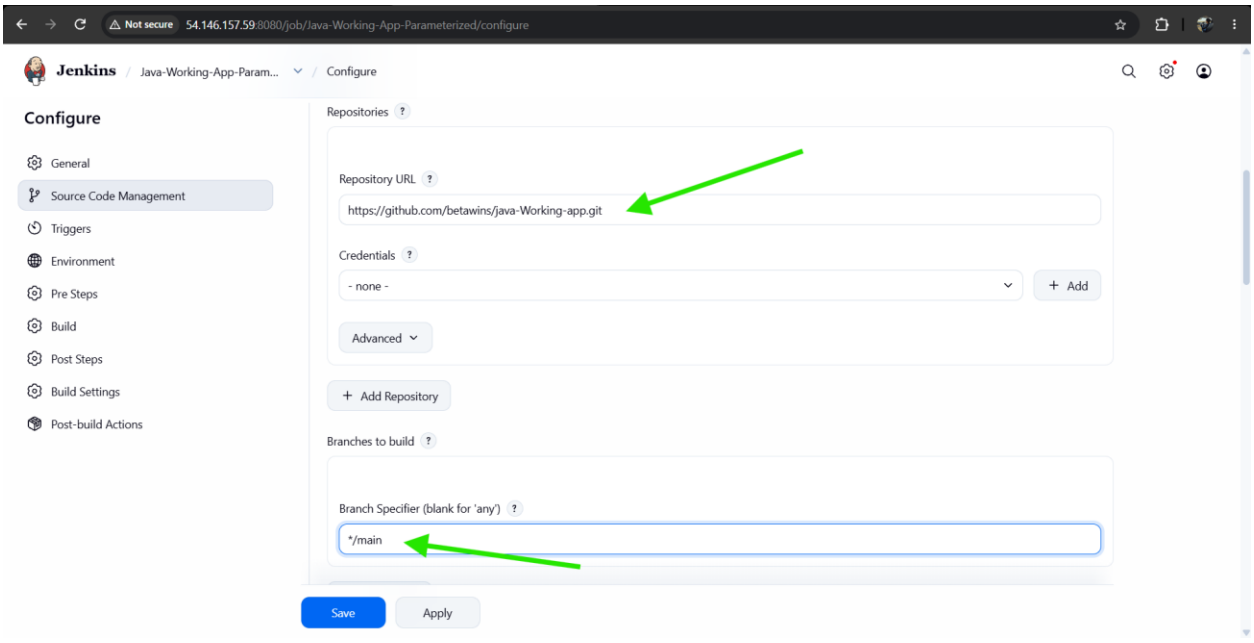
https://github.com/betawins/java-Working-app.git

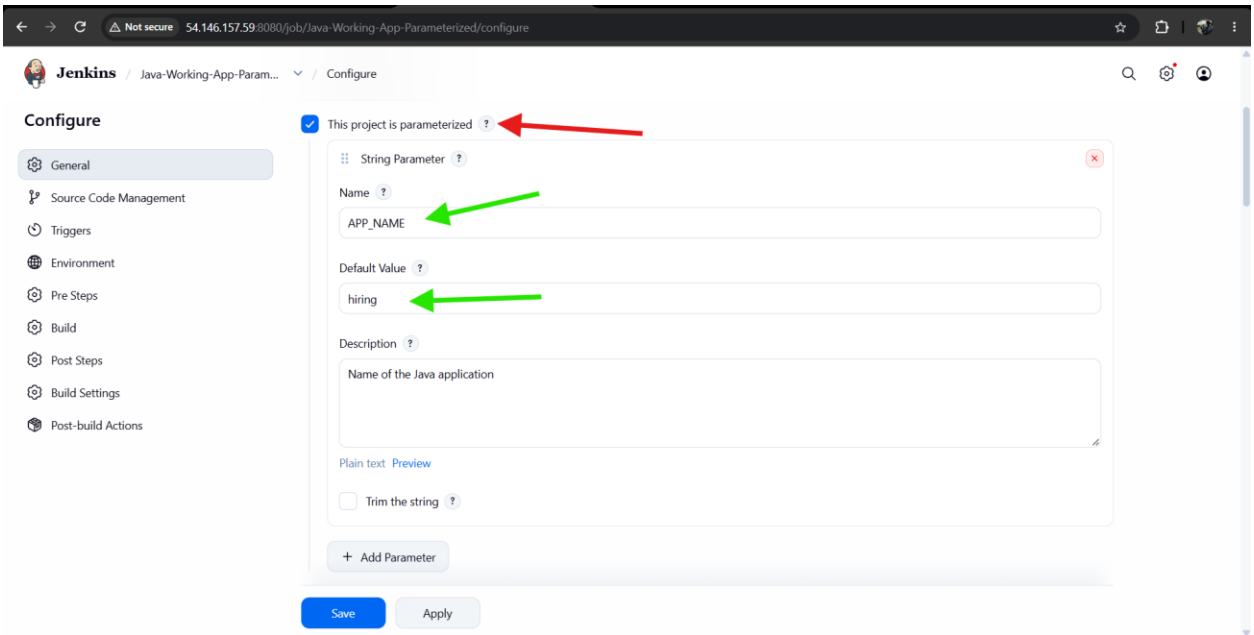
4. Set the branch to build as `*/main`.
5. Enabled This project is parameterized under the General section.
6. Added a String Parameter with the following details:

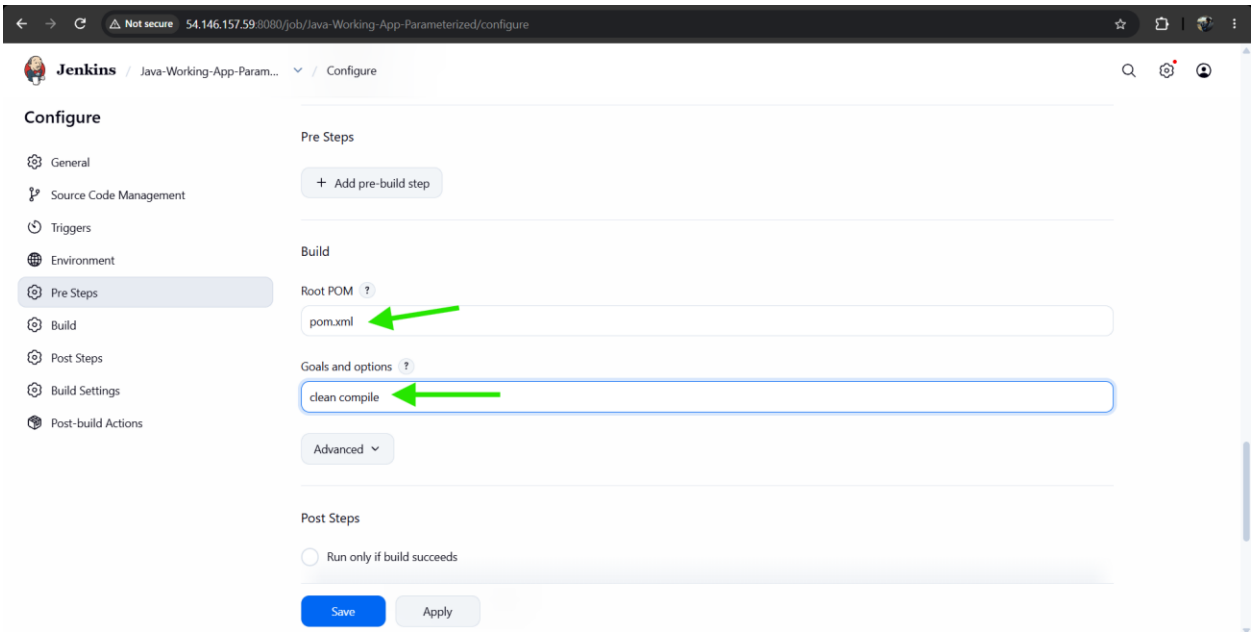
| Field | Value | |---|---| | Name | `APP_NAME` | | Default Value | `hiring` | | Description | Name of the Java application |

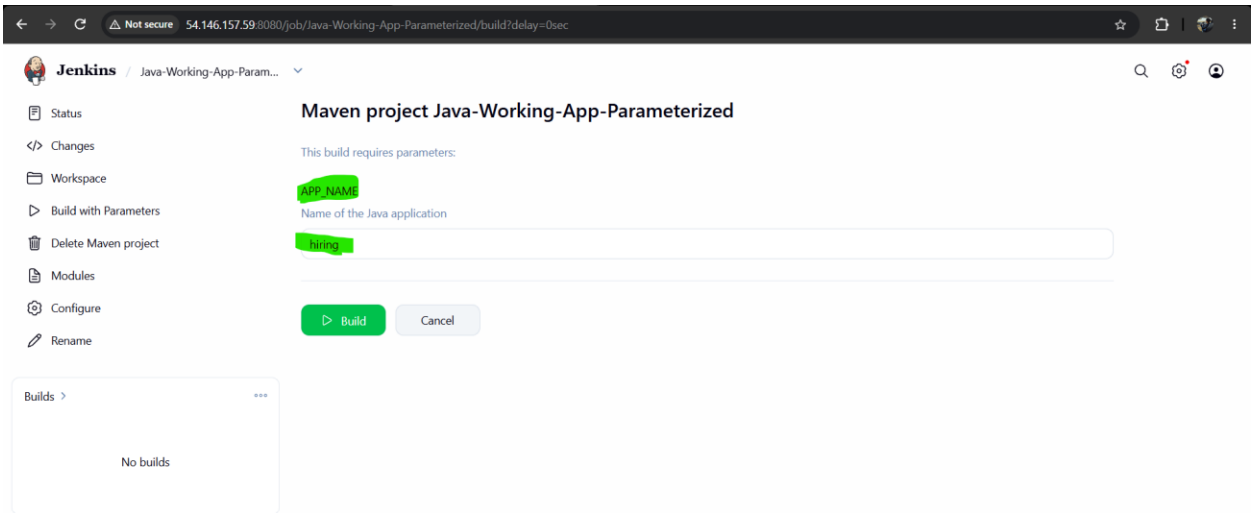
7. Configured the Maven build settings:
    - Root POM: `pom.xml`
    - Goals and options: `clean compile`
8. Saved the job configuration.
9. Selected Build with Parameters, retained the default value `hiring`, and triggered the build.
10. Opened Console Output and verified the build result.

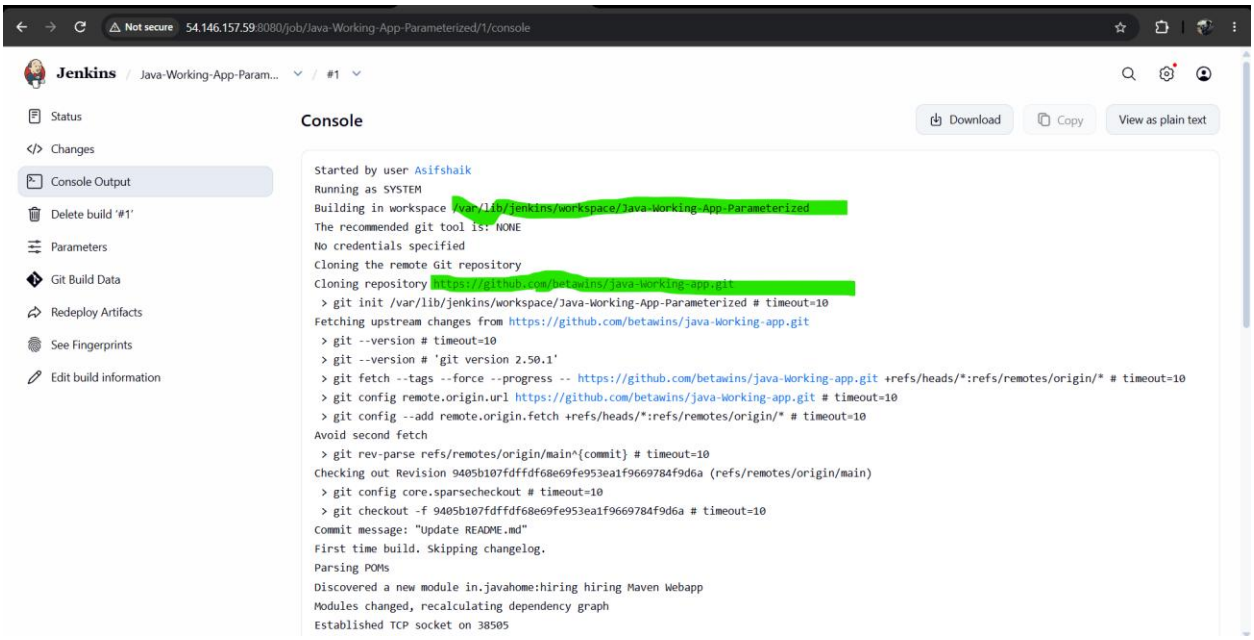

Not secure 54.146.157.59:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Java-Working-App-Parameterized
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
OK

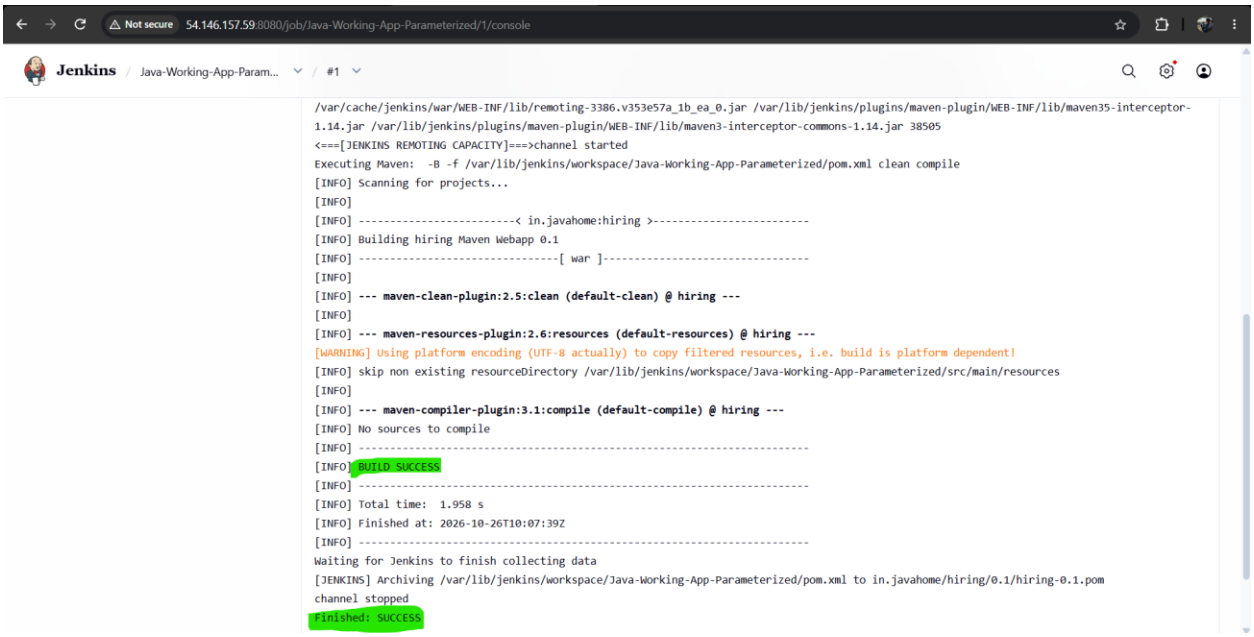

Not secure 54.146.157.59:8080/job/Java-Working-App-Parameterized/configure
Jenkins / Java-Working-App-Param... / Configure
Configure
General
Source Code Management
Triggers
Environment
Pre Steps
Build
Post Steps
Build Settings
Post-build Actions
Repositories
Repository URL
https://github.com/betawins/java-Working-app.git
Credentials
- none -
+ Add
Advanced
+ Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
Save
Apply


Not secure 54.146.157.59:8080/job/Java-Working-App-Parameterized/configure
Jenkins / Java-Working-App-Param... / Configure
Configure
General
Source Code Management
Triggers
Environment
Pre Steps
Build
Post Steps
Build Settings
Post-build Actions
This project is parameterized
String Parameter
Name
APP_NAME
Default Value
hiring
Description
Name of the Java application
Plain text Preview
Trim the string
Add Parameter
Save
Apply


Not secure 54.146.157.59:8080/job/Java-Working-App-Parameterized/configure
Jenkins / Java-Working-App-Param... / Configure
Configure
General
Source Code Management
Triggers
Environment
Pre Steps
Build
Post Steps
Build Settings
Post-build Actions
Pre Steps
Add pre-build step
Build
Root POM
pom.xml
Goals and options
clean compile
Advanced
Post Steps
Run only if build succeeds
Save
Apply


Not secure 54.146.157.59:8080/job/Java-Working-App-Parameterized/build?delay=0sec
Jenkins / Java-Working-App-Param...
Status
Changes
Workspace
Build with Parameters
Delete Maven project
Modules
Configure
Rename
Builds
No builds
Maven project Java-Working-App-Parameterized
This build requires parameters:
APP_NAME
Name of the Java application
hiring
Build
Cancel


Not secure 54.146.157.59:8080/job/Java-Working-App-Parameterized/1/console
Jenkins / Java-Working-App-Param... / #1
Status
Changes
Console Output
Delete build '#1'
Parameters
Git Build Data
Redeploy Artifacts
See Fingerprints
Edit build information
Console
Download
Copy
View as plain text
Started by user Asifshaik
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Java-Working-App-Parameterized
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/java-working-app.git
> git init /var/lib/jenkins/workspace/Java-Working-App-Parameterized # timeout=10
Fetching upstream changes from https://github.com/betawins/java-working-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/java-working-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/java-working-app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
Parsing POMs
Discovered a new module in.javahome:hiring hiring Maven Webapp
Modules changed, recalculating dependency graph
Established TCP socket on 38505




# 3. Observation

- The GitHub repository was cloned successfully.
- The `APP_NAME` parameter was available with the default value `hiring`.
- Maven executed the `clean compile` goals successfully.
- Maven displayed `No sources to compile`, indicating that it did not find Java source files in the configured source directory during this build.
- A platform encoding warning appeared, but it did not fail the build.
- Jenkins archived the `pom.xml` file.

Final build status: `BUILD SUCCESS` — `Finished: SUCCESS`

# 4. Conclusion

Successfully created and executed a parameterized Jenkins Maven project integrated with a GitHub repository. The `APP_NAME` string parameter was configured and made available during build execution. Maven completed the `clean compile` process successfully, demonstrating basic Jenkins job configuration, parameterization, Git integration, and Maven build execution.

## Task 6: What are the global variables in Jenkins?

# 1. Definition

Jenkins global variables are predefined variables that provide information about a Jenkins job, build, workspace, source code, and execution environment. They help automate builds and troubleshoot errors.

# 2. Variables with meanings

### Build variables

| Variable | Meaning |
|---|---|
| `BUILD_NUMBER` | Current build number |
| `BUILD_ID` | Current build identifier |
| `BUILD_TAG` | Unique tag for the build |
| `BUILD_URL` | URL of the current build |
| `BUILD_DISPLAY_NAME` | Display name of the build |
| `CI` | Indicates a CI environment; Jenkins sets it to `true` |

## Job variables

| Variable | Meaning |
|---|---|
| `JOB_NAME` | Name or full path of the Jenkins job |
| `JOB_BASE_NAME` | Job name without its folder path |
| `JOB_URL` | URL of the Jenkins job |

## Workspace and node variables

| Variable | Meaning |
|---|---|
| `WORKSPACE` | Directory where the job's files are stored during execution |
| `WORKSPACE_TMP` | Temporary workspace directory, if available |
| `NODE_NAME` | Name of the Jenkins node running the job |
| `NODE_LABELS` | Labels assigned to the Jenkins node |
| `EXECUTOR_NUMBER` | Executor slot number running the build |
| `JENKINS_HOME` | Directory containing Jenkins configuration and data |
| `JENKINS_URL` | Base URL of the Jenkins server, if configured |

## Operating-system and tool variables

| Variable | Meaning |
|---|---|
| `JAVA_HOME` | Java installation directory, if configured |
| `PATH` | Directories searched for executable commands |
| `HOME` | Home directory of the executing operating-system user |
| `USER` | Operating-system username, if available |
| `SHELL` | Shell executable path, if available |
| `TEMP` | Temporary directory, commonly used on Windows |
| `TMPDIR` | Temporary directory on Unix-like systems |
| `PWD` | Current working directory, if provided by the shell |

## Git variables

These are commonly provided by the Jenkins Git plugin when applicable.

| Variable | Meaning |
| --- | --- |
| `GIT_COMMIT` | Commit hash checked out for the build |
| `GIT_BRANCH` | Branch associated with the checkout |
| `GIT_URL` | Git repository URL |
| `GIT_LOCAL_BRANCH` | Local branch name, if available |
| `GIT_PREVIOUS_COMMIT` | Commit used in the previous build, if available |
| `GIT_PREVIOUS_SUCCESSFUL_COMMIT` | Commit from the previous successful build, if available |

## Pull request and branch variables

These are generally available in supported multibranch Pipeline jobs and depend on the SCM plugin.

| Variable | Meaning |
|---|---|
| `BRANCH_NAME` | Branch being built |
| `BRANCH_IS_PRIMARY` | Whether the branch is marked as primary |
| `CHANGE_ID` | Pull request or change request identifier |
| `CHANGE_BRANCH` | Source branch of the change request |
| `CHANGE_TARGET` | Target branch of the change request |
| `CHANGE_URL` | URL of the change request |
| `CHANGE_TITLE` | Title of the change request |
| `CHANGE_AUTHOR` | Username of the change request author |
| `TAG_NAME` | Git tag associated with the build |

## Pipeline global variables

These are Pipeline objects or variables, not ordinary environment variables.

| Variable | Meaning |
|---|---|
| `env` | Accesses environment variables |
| `params` | Accesses build parameters |
| `currentBuild` | Provides information about the current build |
| `scm` | Represents the configured source control management system in supported contexts |
| `docker` | Provides Docker Pipeline functionality when the plugin is installed |

Common `currentBuild` properties:

| Property | Meaning |
|---|---|
| `currentBuild.number` | Current build number |
| `currentBuild.result` | Build result, which may be `null` before it is set |
| `currentBuild.currentResult` | Current result, such as `SUCCESS` or `FAILURE` |
| `currentBuild.displayName` | Display name of the build |
| `currentBuild.description` | Description of the build |

| | |
|---|---|
| `currentBuild.duration` | Build duration in milliseconds |
| `currentBuild.startTimeInMillis` | Build start time in milliseconds |
| `currentBuild.absoluteUrl` | Absolute URL of the build |

Note: Variable availability depends on Jenkins configuration, plugins, job type, and execution environment.

## 3. Simple real-time example

Suppose your Java application is built using the Jenkins job `Java-Working-App-Parameterized`.

During build number `3`, Jenkins could provide these values:

```
JOB_NAME=Java-Working-App-Parameterized
BUILD_NUMBER=3
GIT_BRANCH=origin/main
WORKSPACE=/var/lib/jenkins/workspace/Java-Working-App-Parameterized
```

You can use them in a Linux shell build step:

```
echo "Job Name: $JOB_NAME"
echo "Build Number: $BUILD_NUMBER"
echo "Git Branch: $GIT_BRANCH"
echo "Workspace: $WORKSPACE"
```

Expected purpose:

- `JOB_NAME` identifies which job is running.
- `BUILD_NUMBER` identifies the current execution.
- `GIT_BRANCH` helps identify the source branch, when available.
- `WORKSPACE` shows where Jenkins is working with the project files.